SOC ANALYST PORTFOLIO CASE STUDY

Detecting an SSH Password-Guessing Attempt

A controlled Wazuh SIEM investigation in a virtualized home lab

RYAN BRYANT | SOC / CYBERSECURITY ANALYST CANDIDATE | AUGUST 2026

OUTCOME Built a two-VM security monitoring environment, forwarded Linux authentication logs to Wazuh, generated a controlled failed-login event, and validated a level 5 alert mapped to MITRE ATT&CK.

Figure 1. Home-lab architecture and telemetry flow.

Portfolio note: This case study documents an authorized lab simulation. No production systems, third-party accounts, or unauthorized targets were involved. AI assistance is disclosed within this report.

1. Executive Summary

I designed and validated a small security operations lab to practice the core workflow of a SOC analyst: collect telemetry, detect suspicious activity, investigate an alert, map it to an adversary technique, and recommend response actions. The environment used Oracle VirtualBox, a Wazuh 4.14.7 SIEM appliance, and an Ubuntu Server 24.04.4 LTS endpoint on an isolated NAT network.

After an official Wazuh package repository returned access errors, I avoided untrusted mirrors and implemented an agentless TCP syslog path supported by Wazuh. I then generated one controlled SSH authentication-failure log entry. Wazuh parsed the event, raised rule 5710 at level 5, and mapped the activity to Password Guessing and SSH in MITRE ATT&CK.

Project dimension	Implementation
Role performed	Lab architect, system administrator, detection engineer, and Tier 1 analyst
Core tools	Wazuh 4.14.7, Ubuntu Server 24.04.4 LTS, rsyslog, Oracle VirtualBox 7.2.16
Network	SOC-Lab NAT network: 10.10.10.0/24
Detection result	1 intentional authentication-failure alert; Wazuh rule 5710; level 5
Framework mapping	MITRE ATT&CK T1110.001 (Password Guessing) and T1021.004 (SSH)
Analyst disposition	True positive, expected/benign due to controlled lab simulation

2. Objectives

- Build a repeatable SIEM lab with a dedicated manager and monitored Linux server.
- Establish network connectivity and centralized log ingestion.
- Generate a safe, controlled authentication-failure event.
- Locate and interpret the resulting Wazuh alert.
- Document triage, scope, MITRE ATT&CK mapping, and recommended response.

3. Lab Architecture and Configuration

Component	Configuration	Purpose
Host	Windows 11 Home; Ryzen 7 9700X; 15.2 GB RAM	Runs the isolated virtualization environment
Virtual network	SOC-Lab NAT; 10.10.10.0/24; gateway 10.10.10.1	Provides VM-to-VM connectivity with controlled outbound access
Wazuh SIEM	10.10.10.3; 8 GB RAM; 2 vCPUs	Receives, parses, correlates, and displays events
Ubuntu endpoint	linux-webserver01; 10.10.10.4; 2 GB RAM; 1 vCPU	Generates and forwards Linux security telemetry
Dashboard access	Host loopback 127.0.0.1:8443 → Wazuh 10.10.10.3:443	Keeps the management interface locally accessible

4. Implementation Workflow

01 VIRTUALIZATION AND NETWORK
Installed VirtualBox 7.2.16, created the SOC-Lab NAT network, imported the official Wazuh OVA, and connected both virtual machines to the same 10.10.10.0/24 segment.

02 WAZUH MANAGER
Validated the Wazuh manager, indexer, and dashboard services; secured dashboard access; created a clean recovery snapshot; and confirmed the manager at 10.10.10.3.

03 UBUNTU ENDPOINT
Installed Ubuntu Server 24.04.4 LTS as LINUX-WEBSERVER01, assigned 10.10.10.4, enabled OpenSSH, and confirmed 0% packet loss to the Wazuh manager.

04 LOG INGESTION
Configured Wazuh to listen for TCP syslog on 10.10.10.3:514 and allow only 10.10.10.4. Configured rsyslog on Ubuntu to forward informational-and-higher messages over TCP.

05 VALIDATION
Checked XML syntax, confirmed the Wazuh manager was active, verified the TCP listener, tested reachability, and then generated a single controlled event with logger.

06 INVESTIGATION
Filtered Wazuh Discover for the unique marker soc-lab-test, expanded the event, reviewed rule metadata, and recorded the analyst disposition and response recommendations.

5. Detection Engineering Details

Wazuh receiver configuration

```
<remote>
  <connection>syslog</connection>
  <port>514</port>
  <protocol>tcp</protocol>
  <allowed-ips>10.10.10.4</allowed-ips>
  <local_ip>10.10.10.3</local_ip>
</remote>
```

Ubuntu forwarding and test event

```
*.info@@10.10.10.3:514
```

```
logger -p authpriv.notice -t sshd "Failed password for invalid user soc-lab-test from 10.10.10.99 port 4444 ssh2"
```

IMPORTANT The message was intentionally created with the Linux logger utility. It simulated a failed SSH login record; it did not execute password guessing or attempt unauthorized access.

6. Evidence: Threat Hunting Overview

The Wazuh Threat Hunting dashboard showed one total alert and one authentication failure. The MITRE visualization associated the event with Password Guessing and SSH, providing the initial lead for triage.

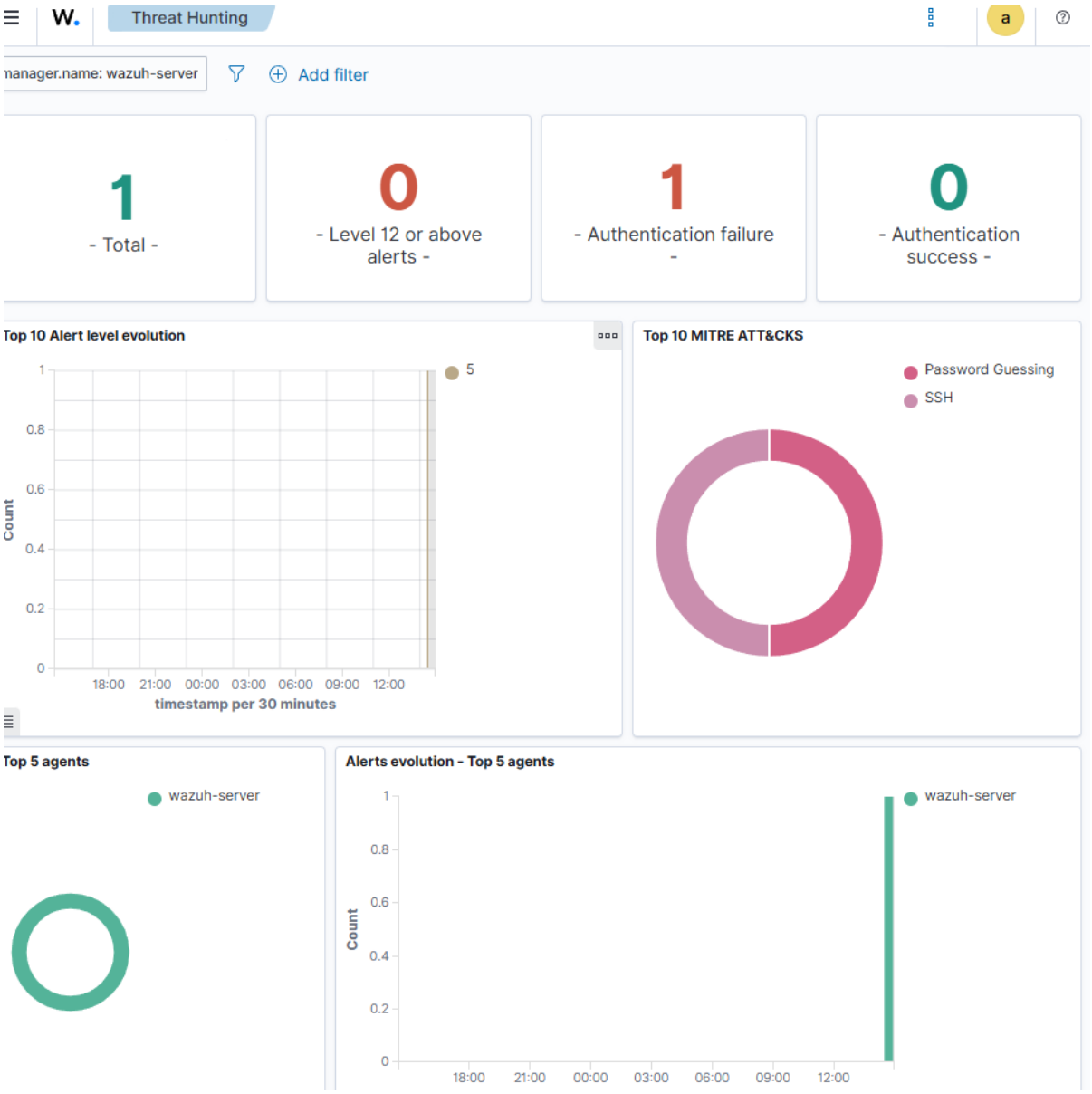

Figure 2. Wazuh Threat Hunting overview: one level 5 authentication-failure alert with MITRE Password Guessing and SSH tags.

Initial analyst observations

- Alert volume was limited to one event, consistent with the controlled test scope.
- The alert was below critical severity but required validation because it involved invalid-user SSH authentication.
- MITRE labels supplied useful context, but the raw fields still needed review before disposition.

7. Evidence: Event-Level Investigation

In Discover, I searched for the unique marker soc-lab-test. The query returned exactly one result. Expanding the record exposed the hostname, rule, event source, classification, compliance mappings, and MITRE fields needed to document the incident.

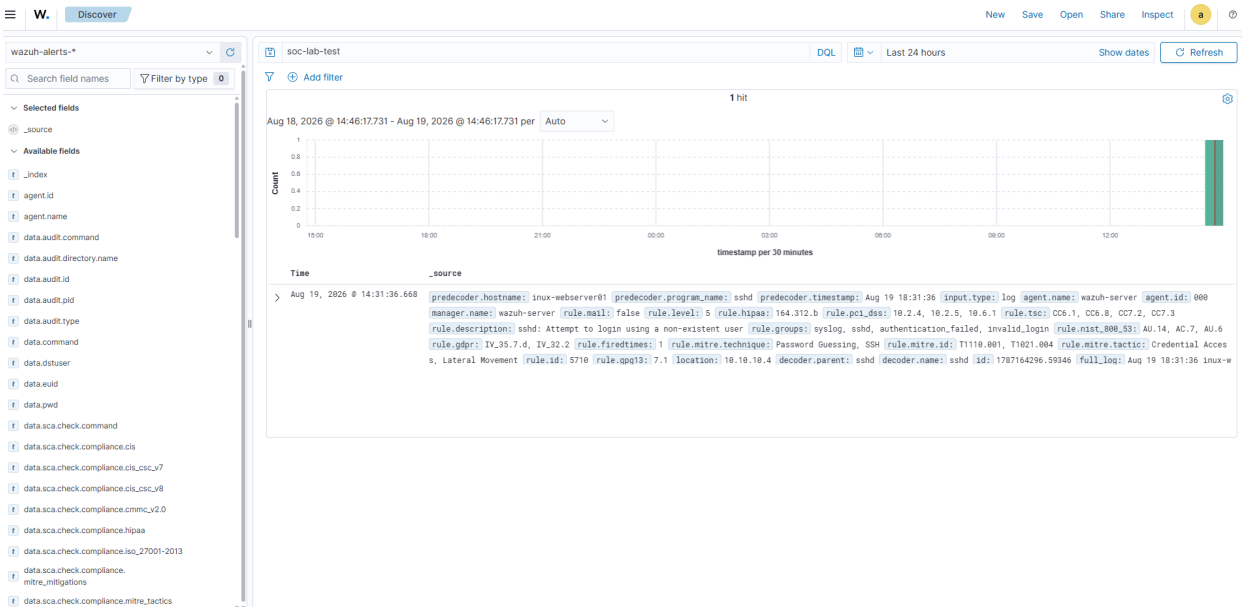

Figure 3. Discover view filtered to soc-lab-test, isolating the single event and its rule and MITRE metadata.

Field	Observed value
Endpoint / location	linux-webserver01 / 10.10.10.4
Program	sshd
Test username	soc-lab-test
Simulated source	10.10.10.99:4444
Wazuh rule	5710 — sshd: Attempt to login using a non-existent user
Severity	Rule level 5
MITRE techniques	T1110.001 Password Guessing; T1021.004 SSH
MITRE tactics	Credential Access; Lateral Movement
Ingestion context	Agentless syslog; agent.id 000 and agent.name wazuh-server are expected

8. Analyst Triage and Disposition

Triage questions answered

Question	Assessment
What happened?	Wazuh received a Linux sshd message describing a failed login for a non-existent user.
Where did it originate?	The log was forwarded from 10.10.10.4. The message contained simulated source 10.10.10.99 and port 4444.
Was access successful?	No. The event describes a failed attempt; the dashboard showed no corresponding authentication success.
How severe was it?	Level 5: suspicious and investigation-worthy, but not evidence by itself of compromise.
Is it malicious?	The detection is technically a true positive for the logged behavior, but the activity is an expected benign simulation.
What is the scope?	One event matched the unique test marker in the selected 24-hour window.

FINAL DISPOSITION TRUE POSITIVE — EXPECTED LAB ACTIVITY. Detection logic and log transport worked as designed. No incident escalation was required.

9. Recommended Response in a Real Environment

- Validate the source IP, asset owner, target account, and time window.
- Search for repeated failures, successful logins after failures, new accounts, sudo activity, and unusual processes.
- Check firewall, VPN, identity-provider, and endpoint telemetry for the same source and username.
- Block or rate-limit a confirmed hostile source; disable or reset a targeted account when warranted.
- Preserve relevant logs, document the evidence and timeline, and escalate if successful access or lateral movement is observed.
- Tune thresholds only after confirming that changes will not suppress meaningful low-volume attacks.

10. Troubleshooting and Engineering Judgment

Challenge	Resolution and lesson
VirtualBox boot stalls and CPU/NMI errors	Reduced vCPU allocations, verified KVM paravirtualization and nested paging, and stabilized both VMs. Lesson: right-size resources and validate the hypervisor path before blaming the guest OS.
Wazuh package repository returned 403 / invalid Release	Declined to use an unofficial mirror. Implemented supported agentless TCP syslog ingestion. Lesson: preserve software-supply-chain trust when troubleshooting.
Malformed Wazuh XML during editing	Restored from a backup, validated XML syntax, restarted the manager, and confirmed the listener. Lesson: back up configuration and test syntax before service restart.
Agentless visibility tradeoff	Documented that syslog supports detection but does not provide full agent capabilities such as file-integrity monitoring and vulnerability inventory.

11. Assistance and Source Transparency

AI assistance disclosure

This project was completed by Ryan Bryant in his personal home lab with assistance from OpenAI ChatGPT (referred to conversationally as “Sarah”). ChatGPT was used as an interactive research, troubleshooting, and writing assistant: it helped interpret error messages, suggest commands and validation checks, locate official documentation, organize the investigation, and edit this portfolio narrative. Ryan personally configured the virtual machines and network, entered the commands, generated the controlled event, reviewed the Wazuh results, captured the screenshots, and verified the final findings. AI suggestions were evaluated against observed system output and the sources listed below; ChatGPT did not independently access or operate the lab computer.

Troubleshooting source trail

Decision or validation	Authoritative basis
VirtualBox stabilization	Oracle VirtualBox documentation for processor settings, nested paging, paravirtualization, and VBoxManage configuration; final settings were validated from the VM information output.
Agentless TCP syslog fallback	Official Wazuh documentation for TCP port 514, allowed-ips, local_ip, Linux rsyslog forwarding, and restarting the manager.
Forwarding syntax and XML checks	Ubuntu rsyslog documentation confirms that @@ selects TCP; Python ElementTree documentation supports parsing an XML file to detect syntax errors; systemd documentation supports service-state checks and restarts.

12. Skills Demonstrated

- SIEM deployment and administration
- Linux system administration
- Network segmentation and TCP service validation
- Centralized syslog engineering
- Alert triage and false-positive/benign-positive disposition
- MITRE ATT&CK mapping
- Evidence-based incident documentation
- Troubleshooting and change validation
- Security-minded supply-chain decision making

13. Employer-Facing Project Summary

RESUME BULLET Built a virtualized Wazuh SIEM lab with an Ubuntu telemetry source, engineered TCP syslog ingestion, simulated and investigated an SSH invalid-user event, and validated a level 5 alert mapped to MITRE ATT&CK T1110.001 and T1021.004.

Interview talking points

- Why I chose an isolated NAT network and local-only dashboard forwarding.
- How I verified each layer: network reachability, service state, listener socket, transport, alert generation, and event fields.
- Why I used a supported agentless fallback instead of an untrusted package mirror.
- How I distinguished a true detection from a real security incident.
- What additional telemetry and containment steps I would use in production.

14. Next Iterations

The next lab phase will install the Wazuh endpoint agent once the official repository path is available, compare agent-based and agentless visibility, generate repeated failures to test correlation thresholds, and create a concise incident ticket with timeline, evidence, scope, severity, and closure rationale.

15. References and Validation Sources

These sources were used to validate the configuration choices, troubleshooting guidance, and technical claims in this case study. Direct system output and the two Wazuh screenshots served as the primary evidence that the lab functioned as described.

OpenAI key concepts for GPT text-generation assistance
Wazuh virtual machine deployment
Wazuh: Configuring syslog on the server
Wazuh: Linux rsyslog log collection
Wazuh alert management
Wazuh brute-force detection proof of concept
Oracle VirtualBox VM configuration and nested paging
Oracle VirtualBox VBoxManage reference
Ubuntu rsyslog forwarding syntax
Python ElementTree XML parser
systemctl service control and inspection